

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: EC2 Gets "Access Denied" While Accessing an S3 Bucket

Interview Question

Your application running on an EC2 instance receives an "Access Denied" error while accessing an S3 bucket. How would you troubleshoot it?

Answer

If an EC2 instance cannot access an S3 bucket, I verify IAM permissions, bucket policies, and encryption settings.

Step 1: Check IAM Role

Ensure the EC2 instance has the correct **IAM Role** attached.

The role should have permissions such as:

- `s3:GetObject`
 - `s3:ListBucket`
 - `s3:PutObject` (if upload is required)
-

Step 2: Check Bucket Policy

Verify the S3 Bucket Policy allows access to the IAM Role.

Also check for any **Explicit Deny** statements.

Step 3: Check Encryption

If the bucket uses **KMS encryption**, ensure the IAM Role has:

- `kms:Decrypt`
 - `kms:Encrypt` (if uploading)
-

Step 4: Test Access

Run:

```
aws s3 ls s3://my-bucket
```

or

```
aws s3 cp test.txt s3://my-bucket/
```

Step 5: Review CloudTrail

Check CloudTrail for **AccessDenied** events to identify which permission is missing.

Real-Time Scenario

A Jenkins server on EC2 couldn't upload build artifacts to an S3 bucket. The IAM Role had **GetObject** permission but was missing **PutObject**. After updating the IAM policy, uploads completed successfully.

Best Practices

- Use IAM Roles instead of access keys.
 - Apply the Principle of Least Privilege.
 - Enable CloudTrail for auditing.
 - Avoid wildcard (*) permissions.
-

Interview Summary

"If an EC2 instance gets an 'Access Denied' error while accessing S3, I first verify the IAM Role, check the Bucket Policy, validate KMS permissions if encryption is enabled, test access using the AWS CLI, and review CloudTrail logs for authorization failures."

Scenario 2: Accidentally Deleted Files from an S3 Bucket

Interview Question

A user accidentally deleted important files from an S3 bucket. How would you recover them?

Answer

The recovery depends on whether **Versioning** is enabled on the bucket.

Step 1: Check Versioning

Verify if **S3 Versioning** is enabled.

- **Enabled** → Files can usually be recovered.
 - **Disabled** → Recovery is only possible from backups or replication.
-

Step 2: Check Object Versions

Open the bucket and enable **Show Versions**.

If the object was deleted, you'll typically see a **Delete Marker** and previous object versions.

Step 3: Restore the File

Recover the object by:

- Deleting the **Delete Marker**, or
- Copying the previous object version to the latest version.

This restores the file without affecting other objects.

Step 4: Check Replication or Backup

If Versioning isn't enabled, verify whether:

- Cross-Region Replication (CRR)
- Same-Region Replication (SRR)
- AWS Backup

contains a copy of the deleted object.

Step 5: Review CloudTrail

Use **CloudTrail** to identify:

- Who deleted the object.
 - When it was deleted.
 - Which IAM user or role performed the action.
-

Real-Time Scenario

A developer accidentally deleted deployment artifacts from a production S3 bucket. Since **Versioning** was enabled, we removed the **Delete Marker**, restoring all files within minutes without restoring from backup.

Best Practices

- Enable **Versioning** for all production buckets.
 - Enable **MFA Delete** for critical buckets (where applicable).
 - Use **Cross-Region Replication** for disaster recovery.
 - Enable CloudTrail for auditing object deletions.
-

Interview Summary

"If files are accidentally deleted from an S3 bucket, I first check whether **Versioning** is enabled. If it is, I restore the object by removing the **Delete Marker** or recovering a previous version. If **Versioning** isn't enabled, I check replication or backups and use **CloudTrail** to determine who deleted the object."

💡 Interview Tip

Common Follow-up Question:

Can you recover a deleted object if **Versioning is disabled?**

Answer:

No, S3 itself cannot recover permanently deleted objects when **Versioning** is disabled.

Recovery is only possible if:

- A backup exists.
- Cross-Region or Same-Region Replication has a copy.
- AWS Backup was configured.

That's why **enabling Versioning is considered a production best practice.**

Scenario 3: Cross-Account S3 Access Is Not Working

Interview Question

An application in AWS Account A needs to access an S3 bucket in Account B, but it receives an "Access Denied" error. How would you troubleshoot it?

Answer

For cross-account S3 access, I verify the IAM Role, Bucket Policy, and permissions in both AWS accounts.

Step 1: Check IAM Role

Ensure the application or EC2 instance in **Account A** is using the correct IAM Role.

The role should have permissions such as:

- `s3:GetObject`
- `s3:ListBucket`
- `s3:PutObject` (if upload is required)

Step 2: Check Bucket Policy

Verify the S3 Bucket Policy in **Account B** allows access to the IAM Role from **Account A**.

Without the bucket policy, cross-account access will fail.

Step 3: Check KMS Permissions

If the bucket uses **SSE-KMS**, ensure the KMS Key Policy also allows the IAM Role from **Account A**.

Having S3 permissions alone is not enough for KMS-encrypted objects.

Step 4: Test Access

From Account A:

```
aws s3 ls s3://my-bucket
```

or

```
aws s3 cp file.txt s3://my-bucket/
```

Step 5: Review CloudTrail

Check **CloudTrail** in both accounts for `AccessDenied` events to identify whether the failure is caused by IAM, the bucket policy, or KMS.

Real-Time Scenario

A Jenkins server in **Account A** needed to upload build artifacts to an S3 bucket in **Account B**. The IAM Role had the correct S3 permissions, but the **Bucket Policy didn't trust the IAM Role**. After updating the Bucket Policy, uploads succeeded without using access keys.

Best Practices

- Use **IAM Roles** instead of access keys.
 - Grant least-privilege permissions.
 - Keep Bucket Policies simple and explicit.
 - If using SSE-KMS, update both the IAM policy and KMS Key Policy.
-

Interview Summary

"If cross-account S3 access fails, I first verify the IAM Role in the source account, then check the Bucket Policy in the destination account, validate KMS permissions if encryption is enabled, test access using the AWS CLI, and review CloudTrail logs to identify the root cause."

💡 Interview Tip

Common Follow-up Question:

Which policy is required for cross-account S3 access?

Answer:

Both are required:

1. **IAM Policy** (Source Account) → Grants the IAM Role permission to access S3.
2. **S3 Bucket Policy** (Destination Account) → Allows the IAM Role from the other account.

If the bucket uses **SSE-KMS**, the **KMS Key Policy** must also allow access.

Scenario 4: S3 Cross-Region Replication (CRR) Is Not Working

Interview Question

You configured Cross-Region Replication (CRR), but new objects are not being replicated to the destination bucket. How would you troubleshoot it?

Answer

If S3 replication is not working, I verify versioning, IAM permissions, replication rules, and encryption settings.

Step 1: Check Versioning

Ensure **Versioning** is enabled on **both** the source and destination buckets.

Replication will not work without versioning.

Step 2: Verify Replication Rule

Check that:

- Replication rule is **Enabled**.
- Correct source bucket, prefix, or tags are configured.

- Destination bucket is correct.
-

Step 3: Check IAM Role

Ensure the replication IAM Role has permissions such as:

- `s3:GetObject`
 - `s3:ReplicateObject`
 - `s3:ReplicateDelete`
-

Step 4: Check KMS Permissions

If the bucket uses **SSE-KMS**, verify the replication role has permission to use the KMS keys in both Regions.

Step 5: Test with a New Object

Upload a **new file** to the source bucket.

Note: CRR replicates **new objects only**. Existing objects are not replicated unless **S3 Batch Replication** is configured.

Real-Time Scenario

A compliance team required all production data to be replicated to another AWS Region. Replication wasn't working because **Versioning was enabled only on the source bucket**. After enabling Versioning on the destination bucket and updating the replication role permissions, new objects were replicated successfully.

Best Practices

- Enable Versioning on both buckets.
 - Monitor replication status using S3 Replication metrics.
 - Use **S3 Batch Replication** for existing objects.
 - Regularly test replication for disaster recovery readiness.
-

Interview Summary

"If S3 Cross-Region Replication isn't working, I first verify Versioning on both buckets, check the replication rule, validate the replication IAM Role, ensure KMS permissions are correct if encryption is used, and upload a new object to confirm replication."

💡 Interview Tip

Common Follow-up Question:

Will existing objects be replicated after enabling CRR?

Answer:

No.

By default, CRR replicates **only new objects** created after replication is enabled.

To replicate existing objects, use **S3 Batch Replication**.

Scenario 5: Application Cannot Upload Large Files to S3

Interview Question

Users can upload small files to S3, but large file uploads fail or timeout. How would you troubleshoot it?

Answer

If large file uploads fail, I check the application configuration, S3 upload method, network, and permissions.

Step 1: Check File Size

Determine whether the issue occurs only with large files or with all uploads.

This helps identify whether it's an application or network issue.

Step 2: Verify Multipart Upload

Ensure the application uses **Multipart Upload** for large files.

Multipart Upload improves reliability and allows uploads to resume if interrupted.

Step 3: Check IAM Permissions

Verify the IAM Role or User has the required permissions:

- `s3:PutObject`
 - `s3:AbortMultipartUpload`
 - `s3:ListBucketMultipartUploads`
-

Step 4: Check Network & Timeouts

Verify:

- Internet connectivity or VPC Endpoint
- Load Balancer or proxy timeout settings
- Application timeout configuration

Large uploads may fail if timeout values are too low.

Step 5: Review Logs

Check:

- Application logs
- CloudTrail
- S3 Access Logs (if enabled)

Look for timeout or permission errors.

Real-Time Scenario

A web application successfully uploaded files smaller than **100 MB**, but uploads of **2 GB** failed. Investigation showed the application was using a single PUT request instead of **Multipart Upload**, causing request timeouts. After implementing Multipart Upload, large files uploaded successfully.

Best Practices

- Use **Multipart Upload** for files larger than **100 MB**.
 - Use **Pre-signed URLs** for direct browser-to-S3 uploads.
 - Configure appropriate application and Load Balancer timeouts.
 - Monitor failed uploads using CloudWatch and application logs.
-

Interview Summary

"If large file uploads to S3 fail, I first determine whether the issue is file-size specific, verify the application uses **Multipart Upload**, check **IAM permissions**, review **network and timeout settings**, and analyze application logs and **CloudTrail** to identify the root cause."

💡 Interview Tip

Common Follow-up Question:

Why should you use Multipart Upload?

Answer:

Benefits of Multipart Upload:

- Faster uploads (parallel uploads)
 - Supports very large files
 - Resume interrupted uploads
 - Better reliability
 - Reduces the need to restart uploads after failures
-

Scenario 6: S3 Lifecycle Policy Is Not Moving Objects

Interview Question

You configured an S3 Lifecycle Policy to move objects to Glacier or Intelligent-Tiering, but the objects are not transitioning. How would you troubleshoot it?

Answer

If Lifecycle rules are not working, I verify the rule configuration, object properties, and bucket settings.

Step 1: Check the Lifecycle Rule

Verify the rule is:

- **Enabled**
 - Applied to the correct bucket
 - Using the correct prefix or object tags
-

Step 2: Verify Object Age

Ensure the objects have reached the configured transition period.

Example:

- Transition to Glacier after **30 days**

Objects newer than 30 days won't move.

Step 3: Check Object Filters

Verify the object matches the Lifecycle rule.

If the rule applies only to:

logs/

Objects stored in:

backup/

won't transition.

Step 4: Check Object Versioning

If Versioning is enabled, verify whether the Lifecycle rule applies to:

- Current versions
- Non-current versions

- Delete markers
-

Step 5: Verify Storage Class

Use the AWS CLI to confirm the current storage class.

```
aws s3api head-object \  
--bucket my-bucket \  
--key file.txt
```

Real-Time Scenario

A company expected application logs to move to **Glacier after 30 days**, but nothing happened. Investigation showed the Lifecycle rule applied only to the **logs/** prefix, while the application stored files in the **app-logs/** folder. After correcting the prefix, new log files transitioned to Glacier as expected.

Best Practices

- Test Lifecycle rules with a dedicated test bucket.
 - Use prefixes and tags carefully.
 - Review Lifecycle rules periodically.
 - Use **Intelligent-Tiering** for unpredictable access patterns.
-

Interview Summary

"If an S3 Lifecycle Policy isn't moving objects, I verify that the rule is enabled, confirm the objects match the configured prefix or tags, check the object age and versioning settings, and validate the storage class using the AWS CLI. Incorrect filters and object age are the most common causes."

💡 Interview Tip

Common Follow-up Question:

When should you use Intelligent-Tiering instead of Glacier?

Intelligent-Tiering	Glacier
Unknown or changing access patterns	Long-term archival
Automatic storage optimization	Lowest storage cost
Immediate retrieval	Retrieval can take minutes to hours (depending on tier)
Frequently changing workloads	Compliance and backup archives

Scenario 7: S3 Static Website Is Not Accessible

Interview Question

You hosted a static website on an S3 bucket, but users receive "403 Access Denied" or "404 Not Found." How would you troubleshoot it?

Answer

If an S3 static website is inaccessible, I verify the bucket configuration, permissions, and website settings.

Step 1: Check Static Website Hosting

Ensure **Static Website Hosting** is enabled.

Verify:

- Index document (e.g., `index.html`)
 - Error document (e.g., `error.html`)
-

Step 2: Check Bucket Policy

Verify the bucket policy allows public read access to the website files.

Without the appropriate policy, users receive **403 Access Denied**.

Step 3: Check Block Public Access

Verify **Block Public Access** settings.

If Block Public Access is enabled, public access will be denied even if the bucket policy allows it.

Step 4: Verify Object Permissions

Ensure:

- `index.html` exists.
- Object names and paths are correct (S3 is case-sensitive).

Example:

`index.html`

is different from:

`Index.html`

Step 5: Test the Website Endpoint

Access the **S3 Website Endpoint** (not the REST API endpoint) and verify the website loads correctly.

Real-Time Scenario

A company's static website returned **403 Access Denied** after deployment. Investigation showed **Block Public Access** was enabled on the bucket. After updating the public access settings and bucket policy, the website became accessible.

Best Practices

- Enable Static Website Hosting.
 - Use the S3 **Website Endpoint** for hosting.
 - Keep Block Public Access enabled for non-public buckets.
 - For production websites, use **CloudFront + S3** instead of exposing the bucket directly.
-

Interview Summary

"If an S3 static website isn't accessible, I verify Static Website Hosting, review the Bucket Policy and Block Public Access settings, ensure the index document exists, check object names, and test the S3 Website Endpoint. Most issues are caused by incorrect permissions or website configuration."

💡 Interview Tip

Common Follow-up Question:

Why is CloudFront preferred over direct S3 website hosting?

Answer:

- Better performance through caching
 - HTTPS support with ACM certificates
 - DDoS protection via AWS Shield
 - Integration with AWS WAF
 - Lower latency for global users
 - Ability to keep the S3 bucket private using Origin Access Control (OAC) or Origin Access Identity (OAI)
-

Scenario 8: High AWS Bill Due to S3 Storage

Interview Question

Your AWS bill has increased significantly because of S3. How would you investigate and optimize the cost?

Answer

If S3 costs increase unexpectedly, I first identify what's driving the cost and then apply the appropriate optimization strategy.

Step 1: Analyze S3 Costs

Use:

- **AWS Cost Explorer**
- **S3 Storage Lens**

Identify:

- Which bucket has increased in size.
 - Which storage class is consuming the most cost.
 - Whether request or data transfer charges have increased.
-

Step 2: Identify Large or Unused Objects

Review:

- Old backups
- Log files
- Duplicate files
- Temporary uploads

Delete objects that are no longer required.

Step 3: Review Storage Class

Ensure data is stored in the appropriate storage class:

- **Standard** → Frequently accessed data
 - **Intelligent-Tiering** → Unknown access pattern
 - **Standard-IA** → Infrequently accessed data
 - **Glacier** → Long-term archive
-

Step 4: Configure Lifecycle Policies

Automatically:

- Move old objects to cheaper storage classes.
 - Delete temporary files after a defined retention period.
-

Step 5: Review Versioning

If Versioning is enabled, check for:

- Old object versions
- Delete markers

Configure Lifecycle rules to remove non-current versions after a retention period.

Real-Time Scenario

A production log bucket grew rapidly because application logs were never deleted. Versioning was also enabled, creating multiple object versions. After implementing Lifecycle policies to transition logs to **Glacier** after **30 days** and delete non-current versions after **90 days**, monthly S3 storage costs were reduced significantly.

Best Practices

- Enable **Lifecycle Policies**.
 - Use **Intelligent-Tiering** for unpredictable workloads.
 - Regularly review **S3 Storage Lens** reports.
 - Clean up incomplete multipart uploads.
 - Remove unused object versions.
-

Interview Summary

"If S3 costs increase, I first analyze the cost using Cost Explorer and S3 Storage Lens, identify large or unused objects, review storage classes, configure Lifecycle policies, and clean up old object versions and incomplete multipart uploads. This helps optimize storage costs without affecting application availability."

Interview Tip

Common Follow-up Question:

What are the biggest contributors to S3 cost?

Answer:

- Large volume of stored data
 - Frequent API requests (GET, PUT, LIST)
 - Data transfer out to the internet
 - Old object versions (Versioning)
 - Incomplete Multipart Uploads
 - Storing infrequently accessed data in the Standard storage class
-

Scenario 9: S3 Event Notifications Are Not Triggering Lambda/SQS/SNS

Interview Question

You configured S3 Event Notifications to trigger a Lambda function (or send messages to SQS/SNS), but nothing happens when a file is uploaded. How would you troubleshoot it?

Answer

If S3 Event Notifications are not working, I verify the event configuration, permissions, and destination service.

Step 1: Check Event Notification Configuration

Verify:

- Correct event type (e.g., `ObjectCreated`)
- Correct bucket
- Correct prefix/suffix filters

Example:

```
Event:
ObjectCreated:Put
```

```
Prefix:
uploads/
```

```
Suffix:
.jpg
```

If the uploaded file doesn't match the filters, the event won't trigger.

Step 2: Verify Destination

Ensure the destination (Lambda, SQS, or SNS) is configured correctly and is active.

Step 3: Check Resource Permissions

Verify the destination allows S3 to invoke it.

Examples:

- Lambda → Resource-based policy allows S3.
- SQS → Queue policy allows S3.
- SNS → Topic policy allows S3.

Without these permissions, S3 cannot deliver the event.

Step 4: Check Logs

Review:

- **CloudWatch Logs** (Lambda)
- SQS queue messages
- CloudTrail (if needed)

Look for permission or invocation errors.

Step 5: Test the Event

Upload a file that matches the configured filters and confirm the event is received.

Real-Time Scenario

A Lambda function was supposed to resize uploaded images. The event notification was configured with the prefix:

`images/`

However, users uploaded files to:

`uploads/`

Since the prefix didn't match, the Lambda function was never triggered. Updating the event configuration resolved the issue.

Best Practices

- Use specific prefix and suffix filters.
- Grant the required resource-based permissions.
- Monitor Lambda and SQS using CloudWatch.
- Test event notifications after every configuration change.

Interview Summary

"If S3 Event Notifications aren't triggering, I first verify the event type and prefix/suffix filters, ensure the destination is configured correctly, check the required resource permissions, review CloudWatch logs, and test by uploading a matching object. Incorrect filters and missing permissions are the most common causes."

💡 Interview Tip

Common Follow-up Question:

Which AWS services can S3 Event Notifications trigger?

Answer:

- **AWS Lambda** – Process files (resize images, validate uploads, etc.)
- **Amazon SQS** – Queue-based asynchronous processing.
- **Amazon SNS** – Send notifications to multiple subscribers.

Note: If you need more advanced routing, retries, or integration with multiple targets, you can use **Amazon EventBridge** with S3 events.

Scenario 10: Design a Secure and Cost-Optimized S3 Architecture for Production

Interview Question

How would you design a secure, highly available, and cost-optimized S3 architecture for a production environment?

Answer

For a production environment, I focus on **security, durability, availability, performance, and cost optimization.**

Step 1: Secure the Bucket

- Block public access by default.
- Use **IAM Roles** and Bucket Policies for access control.

- Enable **Server-Side Encryption (SSE-KMS)** for sensitive data.
-

Step 2: Enable Data Protection

Configure:

- **Versioning** to recover deleted or modified files.
 - **Cross-Region Replication (CRR)** for disaster recovery (if required).
-

Step 3: Optimize Storage Cost

Use the appropriate storage class:

- **S3 Standard** → Frequently accessed data.
- **Intelligent-Tiering** → Unknown access patterns.
- **Standard-IA** → Infrequently accessed data.
- **Glacier** → Long-term archive.

Configure **Lifecycle Policies** to automatically transition old data.

Step 4: Improve Performance

- Use **Multipart Upload** for large files.
 - Use **Amazon CloudFront** to cache static content globally.
 - Use **Pre-Signed URLs** for secure direct uploads from clients.
-

Step 5: Monitoring & Auditing

Enable:

- **CloudTrail** for API auditing.
 - **S3 Server Access Logs** (if required).
 - **S3 Storage Lens** for storage and cost analysis.
 - **CloudWatch** alarms for operational monitoring.
-

Real-Time Scenario

A production application stored customer documents in S3. To improve security and reduce costs, we enabled **SSE-KMS encryption**, **Versioning**, and **Lifecycle Policies** that moved

files older than **90 days** to **Glacier**. We also placed **CloudFront** in front of S3 for faster global access and lower latency. This improved security, reduced storage costs, and enhanced user experience.

Best Practices

- Block Public Access unless the bucket is intentionally public.
 - Enable Versioning for critical data.
 - Use Lifecycle Policies to reduce storage costs.
 - Use CloudFront for content delivery.
 - Grant least-privilege IAM permissions.
 - Enable CloudTrail and Storage Lens for auditing and optimization.
-

Interview Summary

"For a production S3 architecture, I secure the bucket using IAM policies, Bucket Policies, and SSE-KMS encryption. I enable Versioning and, if needed, Cross-Region Replication for data protection. I optimize costs using Lifecycle Policies and appropriate storage classes, improve performance with CloudFront and Multipart Upload, and enable CloudTrail, Storage Lens, and CloudWatch for monitoring and auditing."

✓ S3 Scenarios Completed (10/10)

You now have **10 production-focused S3 scenarios** covering:

1. EC2 gets **Access Denied** while accessing S3.
2. Recover accidentally deleted files.
3. Cross-account S3 access issues.
4. Cross-Region Replication not working.
5. Large file uploads failing.
6. Lifecycle Policy not moving objects.
7. Static website hosted on S3 not accessible.
8. High S3 storage costs.
9. S3 Event Notifications not triggering.
10. Secure and cost-optimized S3 architecture.

These scenarios cover the majority of S3 troubleshooting and architecture questions asked in AWS/DevOps interviews for **4–8 years of experience**.

💡 Quick Revision: Top S3 Interview Commands

```
# List buckets
aws s3 ls

# List objects in a bucket
aws s3 ls s3://my-bucket

# Upload a file
aws s3 cp file.txt s3://my-bucket/

# Download a file
aws s3 cp s3://my-bucket/file.txt .

# View object metadata
aws s3api head-object --bucket my-bucket --key file.txt

# Sync a local directory
aws s3 sync ./data s3://my-bucket/
```
